Chapter 7: Input/Output (I/O) Management

ANSWERS

Exercises:

1. I/O devices can be roughly grouped into three categories. List these categories and given examples for each category.

   - Human readable
     - Used to communicate with the user
     - Printers
     - Video display terminals
       - Display
       - Keyboard
       - Mouse
   - Machine readable
     - Used to communicate with electronic equipment
     - Disk and tape drives
     - Sensors
     - Controllers
     - Actuators
   - Communication
     - Used to communicate with remote devices
     - Digital line drivers
     - Modems

2. There are great differences across I/O devices. Briefly explain the key differences.

   - Application
     - Disk used to store files requires file management software
     - Disk used to store virtual memory pages needs special hardware and software to support it
     - Terminal used by system administrator may have a higher priority
   - Complexity of control
   - Unit of transfer
     - Data may be transferred as a stream of bytes for a terminal or in larger blocks for a disk
   - Data representation
     - Encoding schemes
   - Error conditions
     - Devices respond to errors differently

3. What is Direct Memory Access (DMA)?

   – Blocks of data are moved into memory without involving the processor

- Processor involved at beginning and end only
- Used to avoid **programmed I/O** for large data movement
- Requires **DMA** controller
- Processor delegates I/O operation to the DMA module
- Bypasses CPU to transfer data directly between I/O device and memory -- DMA module transfers data directly to or from memory
- When complete DMA module sends an interrupt signal to the processor

4. The kernel I/O subsystem provides services related to I/O. List and explain these services.

- **Scheduling**
  - Some I/O request ordering via per-device queue
  - Some OSs try fairness
- **Buffering** - store data in memory while transferring between devices
  - To cope with device speed mismatch
  - To cope with device transfer size mismatch
  - To maintain "copy semantics"
- **Caching** - fast memory holding copy of data
  - Always just a copy
  - Key to performance
- **Spooling** - hold output for a device
  - If device can serve only one request at a time
  - i.e., Printing
- **Device reservation** - provides exclusive access to a device
  - System calls for allocation and deallocation
  - Watch out for deadlock

5. What is the distinction between blocking and nonblocking I/O?

- **Blocking** - process suspended until I/O completed
  - Easy to use and understand
  - Insufficient for some needs
- **Nonblocking** - I/O call returns as much as available
  - User interface, data copy (buffered I/O)
  - Implemented via multi-threading
  - Returns quickly with count of bytes read or written

6. List and briefly define three techniques for performing I/O.

- Programmed I/O
  - The simplest form of I/O – the CPU does all the work
  - Processor issues I/O command (on behalf of a process) to an I/O module. Processor has to monitor the status bits

and to feed data into a controller register one byte at a time.
    - Process is busy-waiting for the operation to complete
- Interrupt-driven I/O
    - I/O command is issued, there are two possibilities:
        - Nonblocking: Processor continues executing instructions from the current process. I/O module sends an interrupt when done.
        - Blocking: processor executes instruction from the OS, which will put the current process in a blocked state, and schedule another process.
- Direct Memory Access (DMA)
    - DMA module controls exchange of data between main memory and the I/O device
    - Processor is interrupted only after entire block has been transferred

7. What is the difference between logical I/O and device I/O?

8. What is the difference between block-oriented devices and stream-oriented devices? Give a few examples of each.

- Block devices include disk drives
    - Commands include read, write, seek
    - Raw I/O or file-system access
    - Memory-mapped file access possible
- Character devices include keyboards, mice, serial ports
    - Commands include get, put
    - Libraries layered on top allow line editing

9. How can we improve the performance of our computer system, with regard to I/O management? Give the five strategies.

- Reduce number of context switches
- Reduce data copying
- Reduce interrupts by using large transfers, smart controllers, polling
- Use DMA
- Balance CPU, memory, bus, and I/O performance for highest throughput

10. Why do you expect improved performance using a double buffer rather than a single buffer for I/O?

11. What are the various kinds of performance overheads associated with servicing an interrupt?

Answer: When an interrupt occurs the currently executing process is interrupted and its state is stored in the appropriate process control block. The interrupt service routine is then dispatched in order to deal with the interrupt. On completion of handling of the interrupt, the state of the process is restored and the process is resumed. Therefore, the performance overheads include the cost of saving and restoring process state and the cost of flushing the instruction pipeline and restoring the instructions into the pipeline when the process is restarted.

12. Describe three circumstances under which blocking I/O should be used. Describe three circumstances under which nonblocking I/O should be used. Why not just implement nonblocking I/O and have processes busy-wait until their device is ready?

Answer: Generally, blocking I/O is appropriate when the process will be waiting only for one specific event. Examples include a disk, tape, or keyboard read by an application program. Non-blocking I/O is useful when I/O may come from more than one source and the order of the I/O arrival is not predetermined. Examples include network daemons listening to more than one network socket, window managers that accept mouse movement as well as keyboard input, and I/O-management programs, such as a copy command that copies data between I/O devices. In the last case, the program could optimize its performance by buffering the input and output and using non-blocking I/O to keep both devices fully occupied.
Non-blocking I/O is more complicated for programmers, because of the asynchronous rendezvous that is needed when an I/O occurs. Also, busy waiting is less efficient than interrupt-driven I/O so the overall system performance would decrease.